

A Few GCC Programming Tips

This article explores a few GCC programming tips. It also compares compile time memory allocation and runtime memory allocation, apart from looking at atomic operations in signal handlers, call-back using structure and function pointers, etc.



To begin, note that we are working with GCC version 8.3.0 (Ubuntu 8.3.0-6ubuntu1).

C programming wouldn't be interesting without pointers! The C compiler can allocate memory to a pointer variable during either compile time or runtime. Let us discuss how pointer expressions differ on compile and on runtime memory allocation.

Here's an example for compile time memory allocation for a pointer variable (*//compilealloc.c*):

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
int main()
{
    char *str = "OpenSourceForYou";
    printf("String %s\n",str);
}
```

```
printf("Strlen %lu\n",strlen(str));
printf("Char after pointer
expression %c\n",++*str++);
return 0;
}
```

In this program, memory for the string *OpenSourceForYou* is allocated during compile time.

The expression in *printf* starts evaluating like *++(*str)++* [i.e., precedence of operators is in the order of dereference (*), followed by the pre-increment operator (++) and post-increment operator (++)]. Basically, the pointer expression is trying to increase the ASCII value by 1 for the first character 'O' in the pointer variable, which is supposed to result in the character 'P'.

An example for runtime (dynamic) memory allocation for a pointer variable

is given below:

```
//runtimealloc.c
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
int main()
{
    char *str = (char *)
    malloc(18*sizeof(char));
    strncpy(str,"OpenSourceForYou",16);
    str[16]='\0';
    printf("String %s\n",str);
    printf("Strlen %lu\n",strlen(str));
    printf("Char after pointer expression
%c\n",++*str++);
    return 0;
}
```

In this program, the memory for the string *OpenSourceForYou* is allocated during runtime or dynamic memory allocation.